AI Native 回忆录：稳定性前端 S1 实践复盘

原创 王僖 阿里技术 2026年9月29日 05:02 浙江 12人



这是2026年的第 65 篇文章

（ 本文阅读时间：约 30 分钟 ）

开篇：春天的一个词

2026 年的这个春天，「AI Native」还是一个在集团内部被反复提起、却很少有人真正跑通的词。对多数团队而言，它约等于**「让工程师用上更先进的 AI 工具」**；而对一支既要推进产品交付、又要探索研发范式的小团队来说，它很快变成了一件更具象的事——后端借 AI 写前端页面，产品经理搭可交互原型，设计师把界面直接做成代码，过去这些需要协作才能完成的事情，现在自己就能闭环。提效的同时，问题也开始暴露：

当每个人都能借助 AI 生产，团队如何保证交付质量？

三月份的一次代码 review，把这个问题摆上了桌面。翻开一位后端同学提交的前端 PR，满屏的 tailwind 类名、原生 input配四十行 CSS、手写 SVG，还有直接 fetch 后端接口的组件。功能已经跑起来了，可我却不敢把它们合进代码库里——**以前我们担心的是做不出来。现在，做出来之后的事情却变多了。**

我是一名稳定性团队的前端。团队负责故障、变更、演练、巡检、SLA 等工程平台，团队不大，但是业务很广，同时维护十几个中后台系统以及大量对内工具。过去的半年，在支撑业务需求的同时，我还负责和深度参与两件事：

- **Agentic Ops 智能体应急助手**：让智能体进入应急协作环节，推动产品从 POC 走向真实的应急链路。
- **AI Native 研发模式转型**：探索更高效的协作方式，让不同角色借助 AI 交付的产物，能够被无痛接手、维护和复用。

这是两件互相咬合、逐渐交织的工作：产品迭代越快，对协作会提出更高的要求；规范和工具沉淀下来，又能支撑下一轮交付。本文想要介绍的是，**当更多人开始借助 AI 生产代码，团**

队如何用规则、协作和质量机制，把这些产物纳入真实交付。事情并不是一开始就有清晰的答案，探索过程也并非坦途，感兴趣且容我重头说起。

01
AI，快与慢

> *——每个人都快了，然后呢？*

从去年底开始，团队陆续进入 AI Coding 状态，然而一份代码从生成到上线，中间还有不少工序：对齐组件和接口约定，处理与旧代码的衔接，再交给另一个人评审、接手。AI 生成时省下的时间很容易看见，这些后续工序花掉的成本，却容易被漏算。

能运行，与能接手之间

AI 没有凭空把代码写坏。它只是**沿着自己最熟悉的路径，把用户眼前的问题解决掉了**。

从前端来说，开源语境里常见的 *Vite*、*Next.js*、*Tailwind*、*lucide-react*，很容易成为默认答案。我们线上却同时有 *Ice + Fusion*、*Umi + Ant Design*、*vite + xops*等历史体系。对于一个已有鉴权、请求拦截、主题和组件约定的仓库，默认答案未必是可合入的答案。

风格的不一致只是最容易看见的一层。A 同学做的页面像后台模板，B 同学做的像官网落地页，布局、配色、交互各自成立，摆在一起却不像同一套产品。再往代码里看，搜索框、按钮、图标常常是手写实现，项目里已有的组件库没有被使用。

还有一种更隐蔽的浪费：**原型活不过评审**。PD 用 AI 做出来，大家讨论完，代码就停在会议室里。轮到实现的环节，前端照着它再做一遍。原型的生成成本降了，后续交接方式却没变。前端接到的不是一件快完成的成品，而是一套已经可以演示运行、却还需要重新接入工程体系的实现。**我们开放了代码的生产入口，却没有把生产代码所需的团队判断一起交出去。**

—— 局部的省时，可能变成下一环的加班。

第一条路：教会所有人

我们的第一反应很传统：既然大家开始写前端，就补一门前端基础课。

团队做过系列课程的第一期，讲环境安装、框架、样式、调试。现实很打脸，知识有用，但即使认真学完，仍没人敢大胆动手。因为真正**决定能不能上线的东西，往往在课程之外：**这个接口为什么必须走那层封装，这个组件在哪个版本里有兼容问题，这个老项目为什么不能按新项目的方法装依赖。

一个资深前端接手陌生仓库，会先看什么、排除什么、对哪些细节警惕，这些判断是多年具体工作留下的。让另一个角色先补完同样的经验，再开始跨端，门槛几乎没有降低。

我们想让大家少依赖前端，最后却把条件变成了「先学得像个前端」。

第二条路：把经验都写下来

课程不够，就建知识库。方向看起来很稳妥：把前端知道的规则写下来，让大家查。

做起来才发现，知识库有两处难点：

- **现状没盘清楚**。前后端项目技术栈各异、版本参差，技术债尚未理清，就急着给出统一规范，文档很容易把旧问题固定下来。
- **冲突缺少仲裁**。不同人对「正确写法」各有理解，没有明确 owner 和 review 机制，规则之间会打架。

即使这些问题都解决了，还有最后一步。

用的时候，去哪里查？

需求已经在对话框里，AI 已经准备写代码。我们试图把规范塞进提示词中（彼时 Prompt 方兴未艾 ），妄想让 AI 去学习去判断，但是效果并不好，每次都需要交代一遍不说，太长了就记不住。此外，知识仍然需要有人维护和更新。有冲突的文档直接装进去，矛盾也会跟着进入 AI 的上下文。

第三条路：换一个更完整的平台

我们也接过 Aone Super（集团一站式需求转代码平台），做过几轮 R2C 实测。接入后，几项成本逐渐显现：

- **项目接入**：需要调整原有配置，非一键集成。
- **本地安装**：使用者要装浏览器代理插件，上手有门槛。
- **任务连续性**：长任务偶尔中断，体验不保证。
- **长期维护**：自维护 API Key，成本难控制。

三条路走下来，我们才把问题看清楚了一点：过去所有解法，都发生在 AI 写代码之前。上课、查文档、切平台，都是先要求人完成一个动作，再期待代码符合规范。

能不能把顺序倒过来？——让规则在代码产生的那一刻就到场。

Skill（结构化规则包） 成了我们选择的载体：团队集中维护结构化规则，命中场景时进入 AI 上下文，使用者仍然留在熟悉的工具里做事情。



02
无规矩，不成方圆

> 做木工的人有尺、有墨线，也有些做久了才晓得的门道。哪里要多留一点，哪里不能硬凑，图纸上不一定说得明白；老师傅看一眼，往往就知道下一步要怎么做。麻烦在于，手艺的东西，很难跟着工具一起交出去。徒弟拿到了同一把锯子，也未必能做出像样的木件。

——老师傅手上的判断，怎样交给 AI?

确定用 Skill 之后，我们没有立刻写一篇「前端规范大全」。第一件事是盘点仓库。因为**老师傅拿到东西先看到的，从来不是自己会多少，而是眼前这件活属于哪一套做法。**

手艺虽好，可不要混用

团队前端项目大致落在三类体系里：

分类	技术栈特征	代表平台
经典稳态系	Ice.js 2.x + Fusion + React 16 + Formily	扬灵统一管理、故障管理、变更管控、应急作战室
稳定性标准系	@alife/x-ops-* + ProComponent	AIOps、巡检平台、故障演练、云 SPE、稳定性洞察、案例库
通用新建系	React 18 + Ant Design	内部工具、临时支撑系统

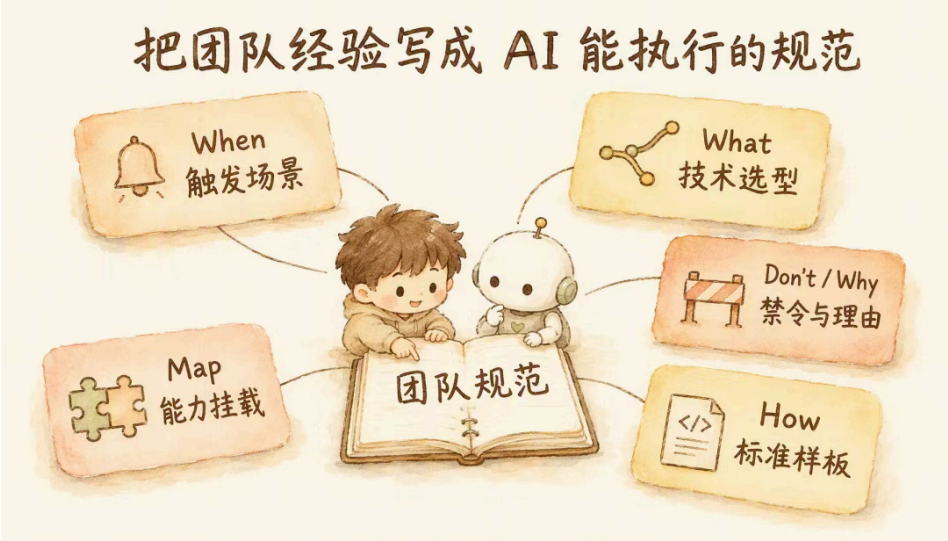
它们的 UI 库、请求库、表单方案和版本约束都不同。一条在新项目里合理的建议，放进老仓库可能恰好有害。Skill 首先要解决「AI 现在在哪个项目里」，然后才轮到「AI 应该怎么写」。

解法很清晰：

> 进入项目 → 识别技术栈 → 确认团队写法 → 查找已有样板 → 按需接入周边能力

把这些判断拆开，就是 `an-frontend-skill` 的五维结构。

- **When，是进场时机。** 只要涉及 React / TSX 的生成、修改、重构或评审，就应加载规范，不必等使用者记住 Skill 的名字再主动调用。
- **What，是选择依据。** 先按项目类型识别历史栈，再对新项目场景给出首选和不推荐项。基于已有工程，减少新的分歧。
- **Don't / Why，是不能碰的边界。** 每条约束后面都写原因。AI 不仅需要知道什么不该做，还需要知道不能做的原因是什么。
- **How，是可以照着做的样板。** 公共组件三段式、API 调用双轨、列表和表单页骨架，几个最小可运行示例，比大段抽象原则更接近真正的交付要求。
- **Map，是去哪里找补充能力。** 设计 Token、国际化 `an-i18n-setup`、项目专属的 `goc-frontend` 和 `x-ops`，主 Skill 不把所有细节重写一遍，命中场景时才加载。



已经跑通的几个场景

Skill 上线后，我们选了四种场景来试：

1. **平台升级改造**。一个中等规模平台，23 个页面、80 多个组件、30 个接口，从 Umi 3 + Ant Design Pro 升级到 Umi 4 + xops-design + ProComponents，用了三天。过去类似改造里，手动对齐规范一度占约 60% 的时间，这部分工作被压到接近零。同期 CFD 演练、SLA 管控、应急作战室 / 直播间等平台也在推进体验与规范统一，整体工期从月级压到周级。去年故障、变更、重保、SPE 四个平台的改造花了三四个月，这次反复对齐规范的工作少了一大块。
2. **AIOps 原型协作**。3 个页面、24 多个组件、8 个核心接口，涉及 SSE、动态数据渲染和钉钉卡片。原型代码的可复用率，从传统协作下不足 30%，到了引入 Skill 与设计直接 AI Coding 后的 **80%**。
3. **国际化改造**。`an-i18n-setup` 把合规化项目的一套流程收成六步自动化流水线和五个配套脚本，每步幂等。FY26 同样的工作量花了约两个月，这一批 Skill 化后**两周落地**。
4. **新建 Status 云产品健康看板**。1 个页面、10 个组件、9 个接口，研发周期三周，AI 代码采纳率 **80%**，已经上线。

Skill 好用了，装 Skill 又成了工作

问题很快从代码里移到了安装环节。

R2C（Requirement to Code）依赖设计规范，设计规范依赖组件知识，一些场景还要 MCP 连接数据。角色越来越多，Skill 越拆越细，一条依赖链少装一环，AI 仍然可以继续工作，只是产出会悄悄走偏。往往事情做完才发现漏了环节，于是现场补装，原本省下的时间又被装配吃回去了。

Plugin 是这个场景下的解药。



Plugin 是 Claude Code 官方给出的插件形态，可以把 Skills、Commands、Agents、Hooks、MCP 放进一个可安装、可分享的扩展单元

集腋成裘：把能力和判断一起打包

我们把这个前端 AI Coding 工作台叫作 **Anchor**。它按三层组织：

层	组成	负责的判断
工作流层	r2c / create-app / api-integration / project-router	下一步做什么
规范层	coding-rules / goc-rules / xops-rules / design-system	这一步应遵守什么
工具层	alidocs / team-info / super-d2c	具体动作需要什么能力或信息

一次需求的主线是：

> R2C 启动 → Project-router 识别仓库 → 加载对应规范 → 执行实现任务

`project-router` 根据 `git remote`、`package.json` 和关键词判断仓库归属。

其中两步按需求触发：

- **新建项目**时调用 `create-app`，已有项目沿用原工程。
- **集成接口**时调用 `api-integration`；工具层也按任务需要取用

分层解决了怎么组织，Hook 和 Command 则补上怎么执行。

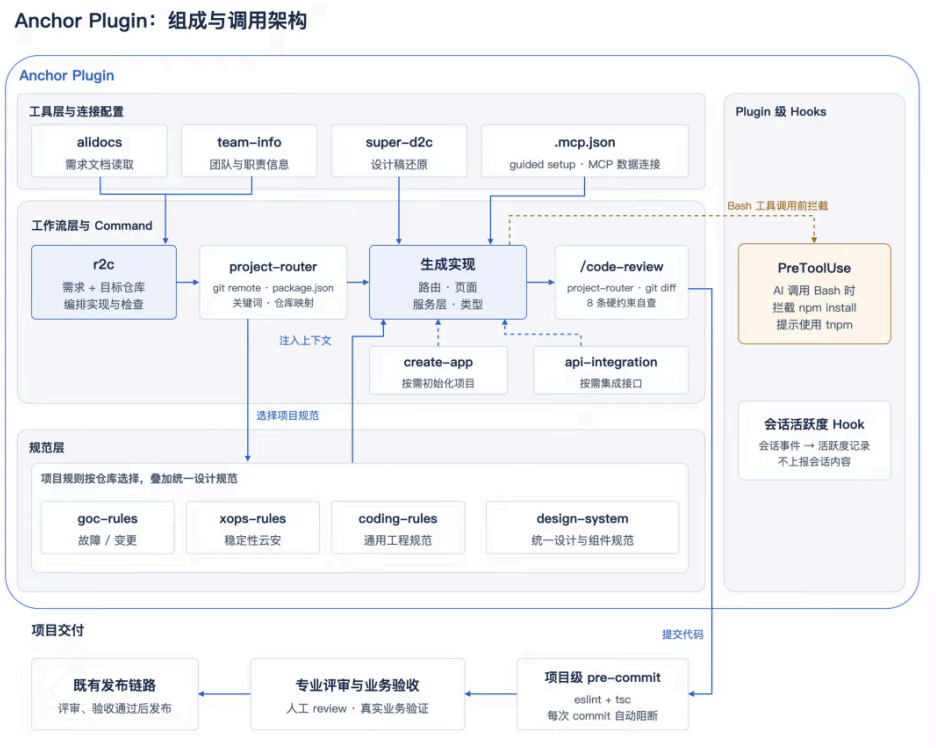
Anchor 在 `PreToolUse` 上挂钩子，AI 调 `npm install` 时直接阻断，另一个 Hook 记录会话活跃度，用于效果度量。

`/code-review` 是显式触发的审查入口：**识别项目 → 获取 `git diff` → 加载对应规范 → 检查变更**

人与 AI 使用同一份检查依据，项目级提交门禁与 Plugin 级的审查、安装拦截，各自负责不同的检查。

检查	放在哪里	何时发生
eslint + tsc	项目级 git pre-commit hook	每次 commit，自动阻断
深度规范审查	Plugin 的 /code-review	提 PR 前，手动触发
npm install 拦截	Plugin 的 PreToolUse hook	AI 调用 Bash 时，自动阻断

至此，原来分别安装六个 Skill、两个 MCP，约十分钟；Plugin 用 `install_from_path` 一次装齐，约三十秒。MCP 从临时发现遗漏，变成 `.mcp.json` 的安装引导；规范从多处冗余，变成插件内的单一事实源。首次 R2C 因缺依赖而失败的情况也明显减少。



03
和而不同

> *——设计和前端的协作，为什么最终没有只留一套代码？*

共用一套代码，不等于共用一种节奏。

我们原来理解的协同，是把大家尽可能放进同一处；真实实践却提醒我们，有些事情先分开，才有机会长期合得上。

把时间拨回到四月，那时的一个念头很有吸引力：**既然设计已经能用 AI 写前端，为什么还要设计一份、前端再还原一份？**

干脆让设计直接改正式代码。省掉翻译，听上去总是对的。

十个环节里，多少时间真的用来做产品

过去的协作链路，我们都很熟悉：

> 需求 → PRD/原型 → 需求评审 → 设计稿 → 设计评审 → 前端还原 → 设计走查 → 修改 → 业务验收 → 上线

十个环节，至少四次交接，每次交接，信息都在衰减，实际运行起来，问题也很明显：

- **信息传递层层损耗：** 需求文档描述的意图，经过设计稿的视觉翻译，再经过前端的代码还原，每一步都在「翻译」，翻译必然引入偏差。
- **等待是最大的浪费：** 设计出稿需要等需求和原型评审，前端开发需要等设计交付，设计走查又需要等前端完成。
- **还原本身就是重复劳动：** 前端花大量时间对着设计稿调间距、对颜色、还原动效，消耗了可观的工时。

我们希望把链路压成这样：

> 需求 → 原型/UI → 原型评审 → 前端调优 → 业务验收 → 上线

设计交付的是可运行、可评审的原型代码。前端把精力放到业务逻辑与工程适配，评审面对的也不再只有静态图片。

一套代码的幻想，只维持了一周

最初，设计直接在我们的正式代码库里改。理想很丰满，代码只有一份，天然就没有还原差异。

一周之内，问题集中冒出来。Tailwind CSS、lucide-react、手写 SVG 等依赖和实现被带进来，与正式库的 x-ops、CSS Modules 冲突；内联 style、上千行单文件、大面积 any，让走查变得很费劲。每一处都可以修，但修完这一处，下一次生成可能又回来。

更难修复的是节奏冲突。

设计常常面向未来多个迭代一起规划，而前端一个 Sprint 只交付其中一部分。两个角色共用同一份代码，发版就要人工拆出哪些属于本期、哪些还在预研，这部分的工作成本比从零还原还高。

我们最终决定：分库。

隔开代码，留下共同的尺度

认清现实后，我们调整策略：让设计独立维护原型代码库，但通过 Skill 来约束 AI Coding 的产出规范。

- 设计独立维护原型库 `aiops-studio`，与正式库 `an-aiops` 物理隔离。
- 技术栈和组件使用由 `frontend-skill` 约束

- 设计系统的颜色、间距、组件用法由 `design-skill` 转成 AI 可理解的要求。

两个库共享同一份规范，不要求共用同一条发布节奏。

拆库以后，最重要的变化不是目录更清楚，而是试错的心理成本降低了。设计、PD、后端都可以在原型库里尝试，暂时跑不起来，也不会影响线上。正式库则可以只接收已经评审、适合当期交付的部分。



设计师交出来的，不再只是一张图

1.🧑‍🎨 设计工作流
MasterGo 设计 → D2C 转码 → 放入原型库→ AI 调整 → 部署评审 → 交付代码

2.👨‍💻 前端工作流
Code Review → 代码移植 → 微调适配

过去两三天的「还原 → 走查 → 修改」，现在可以压到**半天**，省下来的主要是重复翻译。

这套模式**不是银弹**：精细交互仍然会遇到困难。钉钉卡片、复杂图表、交互动效等特殊场景，设计通过 AI Coding 还难以稳定完成，这部分我们继续走「设计出稿 → 前端实现」的路径，约占 20%。剩下约 80% 可以走「设计交付代码」的新链路。

VersionShell：让上周的想法还能被打开

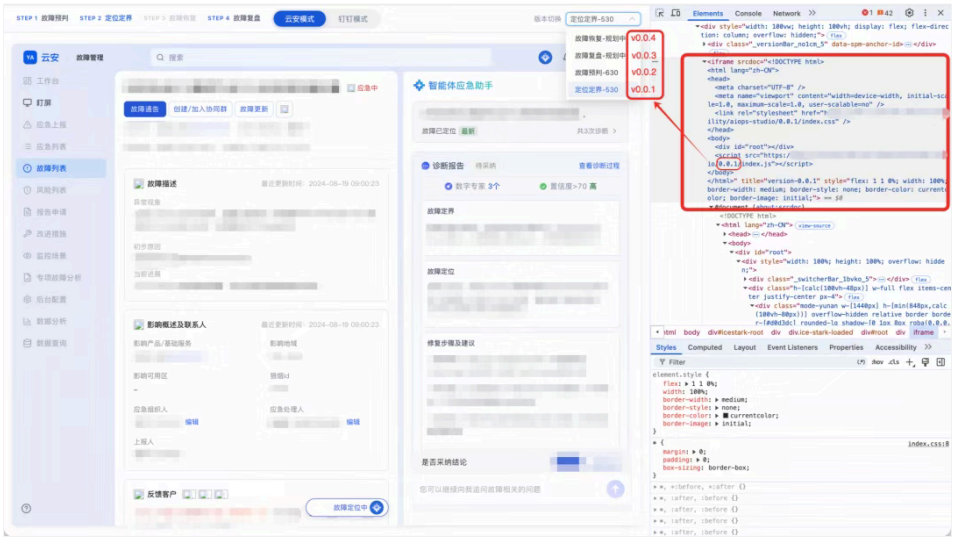
分库之后，原型里的协作也并不天然有序。前端、设计、后端分别推进多个需求，大家的评审时间不同、迭代周期不同、交付节奏也不同。如果所有分支都部署到同一环境，A 的最新改动可能会直接覆盖 B 还没评审完的版本。

为了做到 **「版本隔离、互不干扰、而且在线上环境可以随时切换查看任意模块的原型状态」**，我们设计了一个轻量的版本管理工具 VersionShell，多人独立交付的流程变成：

> 各版本独立构建 IIFE 包 → 发布到 CDN → VersionShell 按需加载指定版本

当被问到**「上周那版是怎么设计的？」**前端不必重新部署，切一下下拉就能回到那版。

当原型可以被指定、被分享、被比较，它才真正开始具备代码资产的生命周期——**评审会不再是原型的终点。**



可复用率，是怎样一点点长上来的

三个月留下的记录，比对新流程的描述更有说服力：

指标	aiops-studio 原型库	an-aiops 正式库
总提交数	104+ commits	399+ commits
贡献者/提交数	前端 34 + 设计 50 + 后端 20	前端
活跃周期	2026.04 — 至今	2026.03.02 — 至今
已发布版本	4 个（v0.0.1 ～ v0.0.4）	保持稳定的双周迭代
覆盖模块	定位定界、故障预判、故障复盘、AI 聊天助手、**变更查询**	**智能诊断**、定位定界、故障预判、故障复盘、AI 聊天助手

回头看，**「一套代码」**的设想并非毫无道理。它想省掉重复劳动，目标是对的；但它低估了不同角色工作节奏的差异。分库隔离后，试错的成本降到足够低，协作的形态也随之演化，这套模式已经逐步扩散到团队内外的不同角色，在新的「**xx 即代码**」的场景下继续演进。



04
君子不器

> ——当一个人能做更多事，角色的边界在哪里？

PDFE：实践在前，定义在后

三月初，**Agentic Ops 智能体应急助手**启动 POC，团队短小精干：**1 前端 + 4 后端，没有 PD，也没有设计师**。项目要与神农、开放平台等多方协作，从零构建智能体应急的 Harness。

作为前端，没有人告诉我，**「从今天开始你要用 PDFE 的角色来干活」**，是项目的形态，将前端推到了这个位置。



AI 产品设计前端工程师 Product · Design · Frontend Engineer

维度	传统产研团队	AI 时代的 PDFE
角色构成	产品经理、设计师、前端工程师	三合一的复合型角色
核心职责	各自负责需求、设计、开发环节	负责从业务意图到用户界面的全链路
协作模式	多角色串行协作，沟通链路长	独立闭环，大幅减少沟通成本
关键能力	单一领域的专业技能深度	产品思维、设计品味、前端工程能力

一行代码，三种判断

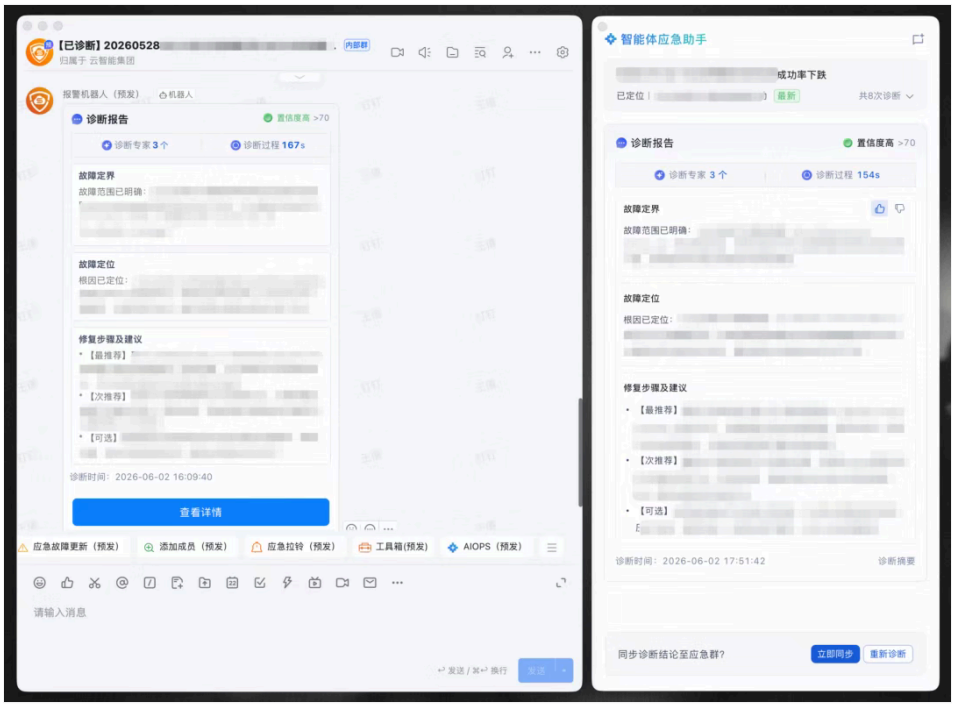
多 Agent 推理可视化，是这种工作状态的一个缩影。

工程上，可以把流式内容全部铺开，也可以在完成后折叠。为什么最后选择后者，不能只从组件实现出发：应急时 SRE/GOC 先要结论，推理过程又需要可以展开审查。于是，**产品上的信息优先级、设计上的一屏呈现与可展开细节，最后落到前端的折叠和流式渲染策略上。**

SSE 协议演进也是类似的问题：

- 后端协议升级，不应该让用户感觉页面反复变化；
- 前端的渲染逻辑，也不该与每一种历史事件格式绑死。

工程实现的背后同时有对**用户体验**和**交互稳定性**的判断。



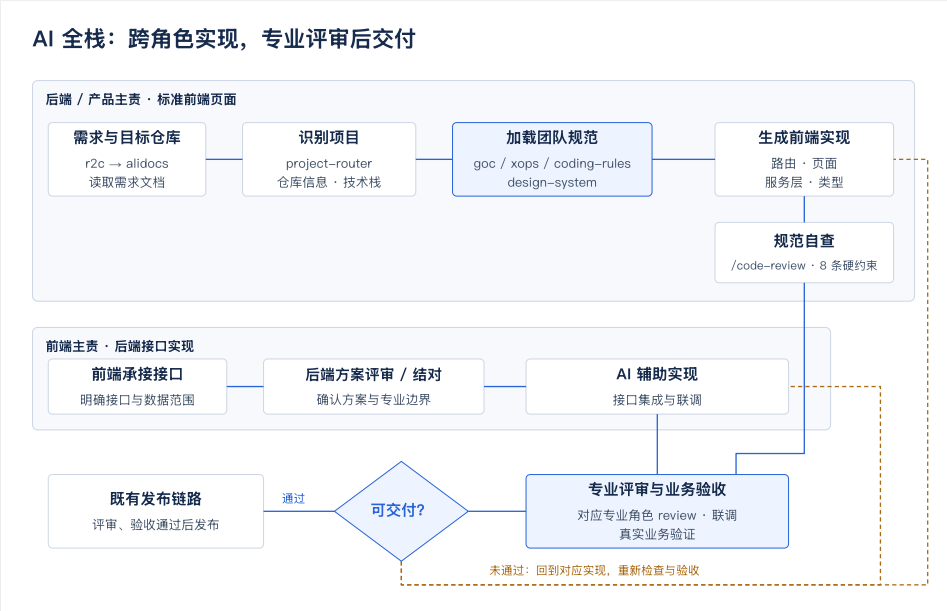
一个人能补位，一群人呢?

PDFE 讲的是**一个角色实现跨职责**；到了七月，团队全栈转型启动，事情又往前走了一步：**不同角色开始进入彼此的实现领域，前后端的日常分工也在慢慢变化。**

接到需求后，我们会分析需求的特征，一些标准化的页面需求，后端可以借助 R2C 能力快速将页面做出来；一些相对轻量的业务逻辑，前端也能在 AI 辅助下写接口，遇到没把握的地方找对方一起看

> *——实现上不再严格按前后端切开，但代码评审仍由对应的 owner 把关。*

少一些来回交接，双方也就有更多时间处理架构、性能和复杂业务问题。



工具降低了门槛，却累积了债务

熟悉一个页面的业务，并不意味着熟悉另一端的事务边界。界面上看不到失败时的脏数据，也看不到半年后某条查询会不会拖慢服务。不熟悉的语言被 AI 写出来以后，人很容易把**「我看到了结果」**误当成**「我掌握了过程」**。

全栈是一件需要考虑 ROI 的事情。

结合团队实践，如实陈述我们感触最深的有几个问题：

1. **需要结对编程**。前端很难 100% 独立完成后端实现，服务间调用链、数据一致性等判断仍离不开后端支持。AI 生成的方案与代码，也需要经过后端的严格评审。
2. **无法准确排期**。写代码本身可能很快，但是环境配置、联调、发布、排障，却是陌生领域里最难填坑，如果对流程不熟悉，工期就很难准确评估。
3. **代码评审成本高**。提交变多，而 reviewer 的时间没有同步增加。前端的布局与交互问题，需要打开浏览器检查；后端方案里的架构问题，也需要相应的专业判断。
4. **维护成本高**。跨端实现的人不一定长期维护，接手的人又没参加当初的生成过程。上线时没有明确维护人，这笔技术债就可能留到下一次改需求。

对此，我们尝试的是**有条件、有选择的全栈**。

适用边界，也应该成为交付的一部分

对 PDFE，适用前提是**探索期、专业角色暂缺、已有规范与业务积累**。成熟产品的版本迭代、高视觉要求或复杂企业级需求，仍需专业分工。

对 AI 全栈，可以按需求结构判断：

- **标准中后台页面**：以 CRUD 和表单为主，后端 / PD 能承接实现，收益主要来自减少排期等待。
- **前后端联动需求**：业务逻辑中等、排期和优先级可控，收益主要来自缩短沟通链路，同时需要投入结对和评审。
- **核心链路与复杂问题**：性能优化、数据一致性等仍由相应领域专家主导，不能只按代码生成速度估算成本。

具体来说，一个需求要不要跨端，需要**先评估复杂度和优先级**，看这笔账是否划算。同样一件事，对熟悉相关技术的同学可能顺手就能完成，换个人却未必。如果跨端过程中，经验能沉淀下来并实现复用，那就值得投入。

所以，全栈建议做且最有价值的事，是把实践的过程做成回路：

> 目标 → 上下文 → 工具接入 → 多轮迭代 → 验证 → 沉淀

这样，当项目收尾，我们回首这段跨端经历时，不会因没有留下记录而遗憾，也不会因埋下技术债务而不安，而能够说：这一路的摸索，已经沉淀成团队可以复用的方法——**让下一个接手的人，少走一些弯路**。



05
授人以渔

> —— *事情做完，除了结果还能留下什么？*

个人的效率再高，也很难变成团队的能力。

在个人的实践同时，我也一直在思考，可以给团队和组织带来什么。

ai-drawio 流程图助手：从 Prompt 到 App 再到 Skill

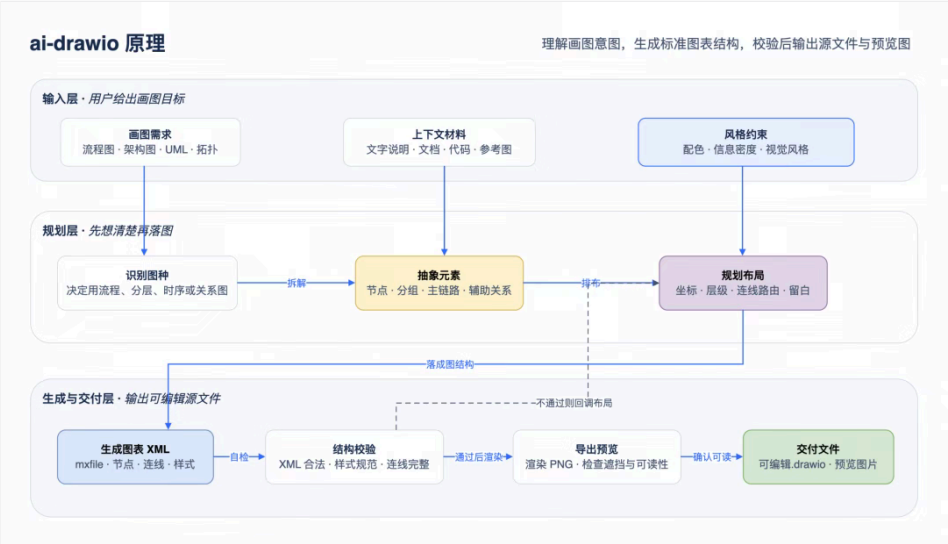
架构图、流程图是研发几乎每天都在接触和产出的东西。最早，我们的解法是通过一套 Prompt 模板，让 AI 生成 XML，需要时复制一份。

后来做成 Web App，叫 Andraw，有独立界面，也更像一个完整产品。

从产品形态上看，这是往前走了一步。然而独立 App 还需要处理**算力归属、自维护 API Key** 等问题。工具做落地了，**运营和维护**却成了新的问题。这与第一章接 R2C 平台时的阻力很像。我们不断想给用户一个新入口。于是 Andraw 的核心能力被沉淀成 ai-drawioskill。用户在 **Qoder / 千问办公**里说一句话，就可以产出标准的 .drawio 文件。

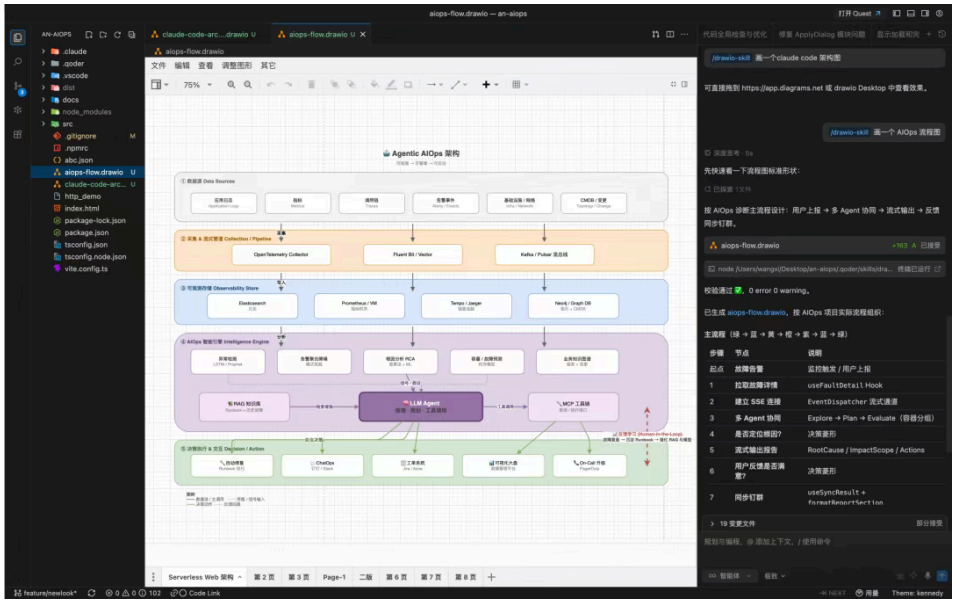
画图能力直接进入已有的工作会话，使用者不必再为这一个动作专门切换平台。本文中架构图与流程图，也属于这套画图能力的实际产物。相比「又做成一个 App」，这类看得见的复用更接近我们想要的结果。

实现原理

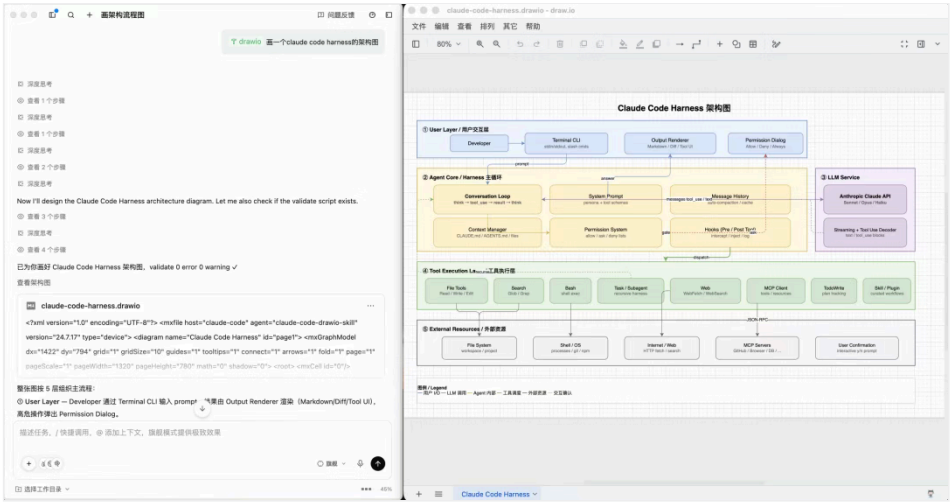


效果展示

Qoder：“画一个 AIOps 流程“



千问办公：“画一个 Harness 架构图”



Andraw Web / Mac App：“画一个 OpenClaw 架构图”



Q萌手绘（q-sketch）：画风也可以成为公共解法

文章数量增加之后，我发现：

1. 同一组专题里，有的图来自 AI，有的来自网络，有的自己画。单张看都没有问题，放在一起，色调、笔触和人物的比例却**很难对齐**
2. 即使是同一篇文章，让 AI 分别生成几张，**风格也容易前后不一**
3. 流行的 Notion 小黄人配图虽然效果不错，但放到自己的文章里，又**少了点辨识度**

于是，我把一种更适合**技术博客、知识分享、团队协作**类文章的手绘风格整理成了 Skill，角色形象、色彩、Prompt 模板被写进了规则。

- 铅笔线稿、暖色水彩、Q 版人物
- 覆盖前端、后端、产品、设计、业务、测试六类角色
- 搭配暖橙、浅绿、暗夜暖金三种主题

可爱圆润的 Q版卡通人物 + 铅笔线稿底子 + 暖色水彩上色，营造轻松亲切、专业但不冰冷的视觉感受。



效果展示（本文配图由本 Skill 辅助生成）



ata-article：把反复回答的写作问题收进 Skill

文章发得多了，大家经常会来问我怎么写好技术文章。参考之前的写作经历，我把自己整理内容时用到的方法整理成了技能，梳理出一份流程：

> 明确读者与选题 → 列大纲（背景、动作、结果、展望） → 填充事实与正文 → 核对数据 → 配图 → 审查语言与表达

具体要检查什么，也一起写进了 Skill：

- **选题和结构**：明确要讲清楚哪件事，把问题、选择、实现和结果连起来
- **事实和过程**：写清做过什么、卡在哪里、为什么换方案，用具体动作和数据支撑判断
- **配图和表达**：配图跟着内容走，删掉概念堆砌、营销腔和空泛结论
- **AI 味自检**：给出四类反模式（概念搬运、套壳实践、PPT 扩写、强行关联）

核心规则

01　写作哲学	02　叙事组件
讲清楚一件事是怎么做成的 • 少陈述价值观，多写具体动作和踩坑记录 • 承认边界 • 数据要朴素要具体 • 重点要加粗，术语要解释	好的框架可以让文章立得住 • 背景锚点 / 遇到什么困难 • 关键动作 / 如何解决的 • 数据量化 / 拿到了什么结果 • 反思与规划 / 有什么局限、未来怎么突破
03　AI 味自检	**04　文案规范**
对照 AI 味清单逐条过一遍 • "不是 A，而是 B" • 滥用 Emoji 表情符号 • 大段列表 + 表格堆叠 • 大量断言和结论，无论证过程和推理链条	逻辑要通顺，职责要清晰，重点要突出 • 一段只讲一件事 • 长段落拆成"短句 + 列表 + 短句" • 重要动作或结论独立成行加粗 • 数据呈现给出绝对值的同时，给出比例或对比基准

借助这套流程，大家就可以直接拿实践材料让 AI 起草，不必绞尽脑汁想文案，也可以避免 AI 味。

AI 故障查询助手：一周落地，背后已经不是从零开始

六月底，我们前后端**两个人**用 **OneAgent**（团队自建的 AI 能力底座） 与 **XOps**（运维域前端组件），一周跑通了故障查询 AI 助手的全流程。没有造轮子，而是组装现有能力工具链覆盖了几部分工作。

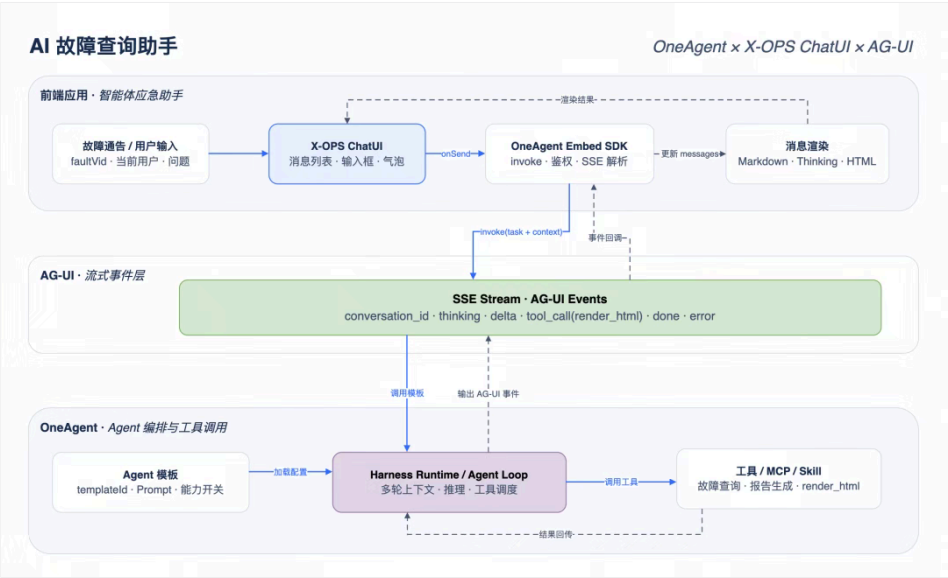
过去，**什么都要重头造**：改一次就联调一轮

- 会话界面从零手搭
- 事件协议一条条对齐
- 编排调度后端自己实现

现在，**只做编排**：高效落地的同时，保证规范和体验

- X-OPS ChatUI：标准 Chat 组件，提供会话骨架
- OneAgent：AI 能力底座，承接编排
- AG-UI：智能体与前端 UI 交互协议

实现原理



效果展示（这套 OneAgent + X-OPS ChatUI + AG-UI 组合，已成为团队后续新 AI 业务应用的默认脚手架）



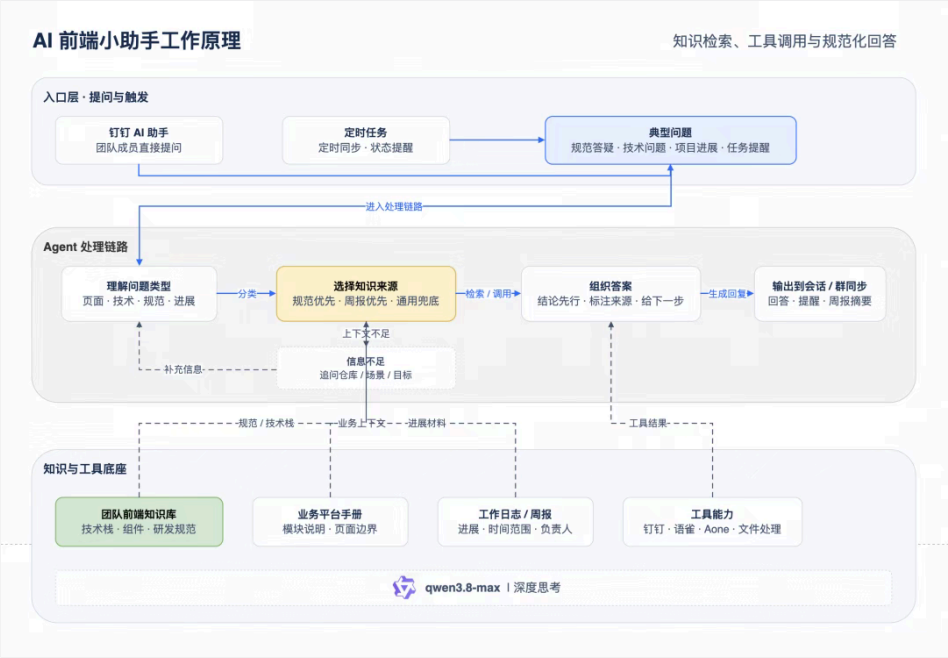
AI 前端小助手： 知识不只在写代码的时候才有用

「前端 AI 小助理」，一个基于集团 AI Studio 配置的智能 Agent，内置了钉钉、语雀、Aone、消息通知、文件处理等工具能力，目前集成在阿里钉 AI 助手里。它的工作范围分四类：

- **团队规范答疑**：回答团队前端项目的技术栈、组件与规范问题。
- **技术问题解答**：覆盖 React、TypeScript、Ice.js、Vite、微前端等技术。
- **项目进展查询**：依据知识库里的周报、工作日志等材料回答，并标明时间范围。
- **定时任务提醒**：每周五上午十点同步到团队成员，更新 Aone 状态，并根据需求进展自动生成周报。

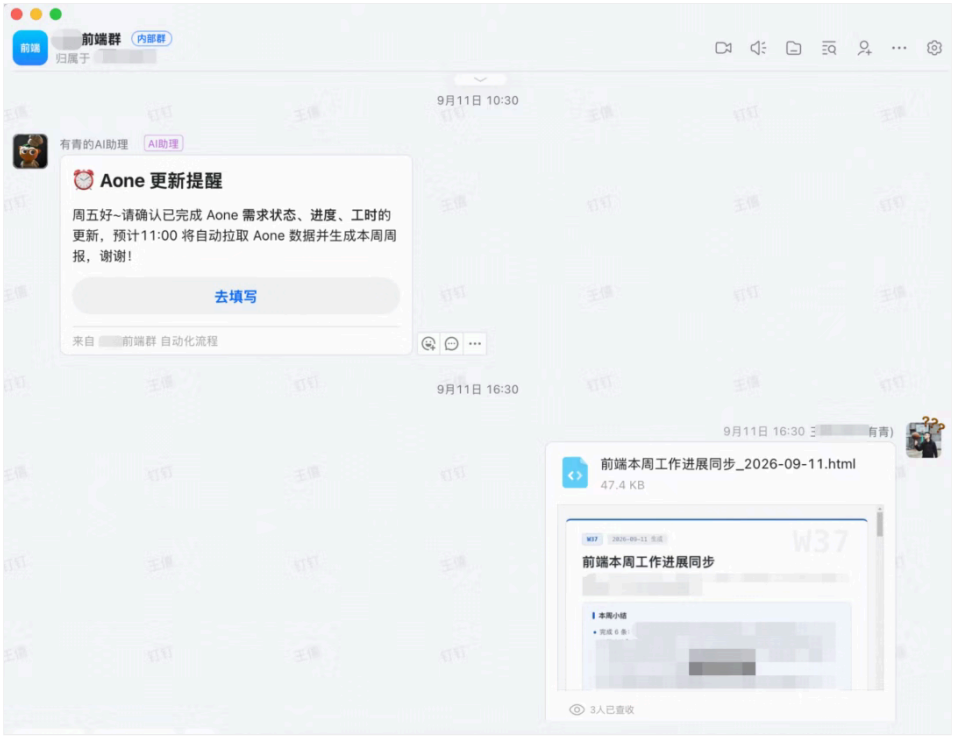
问答能力背后接的是几份知识来源：**团队前端知识库、各业务平台产品手册、工作日志、前端周报**。

实现原理


AI 前端小助手工作原理
知识检索、工具调用与规范化回答
入口层 · 提问与触发
钉钉 AI 助手
团队成员直接提问
定时任务
定时同步 · 状态提醒
典型问题
规范答疑 · 技术问题 · 项目进展 · 任务提醒
进入处理链路
Agent 处理链路
理解问题类型
页面 · 技术 · 规范 · 进展
分类
选择知识来源
规范优先 · 周报优先 · 通用兜底
检索 / 调用
组织答案
结论先行 · 标注来源 · 给下一步
生成回复
输出到会话 / 群同步
回答 · 提醒 · 周报摘要
上下文不足
信息不足
追问仓库 / 场景 / 目标
补充信息
规范 / 技术栈
业务上下文
进展材料
工具结果
知识与工具底座
团队前端知识库
技术栈 · 组件 · 研发规范
业务平台手册
模块说明 · 页面边界
工作日志 / 周报
进展 · 时间范围 · 负责人
工具能力
钉钉 · 语雀 · Aone · 文件处理
qwen3.8-max | 深度思考

效果展示


有青的 AI 小助理
稳定性前端知识专家 · 随时在线的资深前端同事
基于 IdeaLab · 深度思考模式
解答前端页面实现、技术选型、研发规范、组件归属、项目结构，以及团队分工与工作进展；回答先给结论，再给依据和下一步。
结论先行 · 有据可查 · 不臆造 · 信息不足先追问
团队规范答疑
前端技术解答
项目进展查询
周五定时提醒与周报



06
朝花夕拾

> *——这几个月，我们证明了什么？*

已经发生的变化

这是一份 **2026.07** 的稳定性团队快照，覆盖前后端等角色：

- **AI Coding 渗透率**：100%
- **AI 采纳占比**：79.6%
- **AI 辅助代码量**：1,216,288

一些更细的记录也说明 AI 的**使用深度**：

- 单人单月 AI 采纳代码最高 **36,554 行，占比 95.66%**。
- 有人一个迭代 **19** 个需求里，**13** 个由 AI 辅助，产生 **165** 次提交。
- 有人负责的六项需求，AI 覆盖率达到 **95%**。
- 有 **4** 人整个 S1 都在实践 **100%** 端到端的交付

AI 已经进入日常生产，而非少数人的试验。

感觉快，与真的快之间

几项外部研究提醒我们，主观感受与实际结果之间可能有差距：

- Sonar 调查：38% 的开发者认为审查 AI 代码比审查人类代码更费精力
- METR 2025 年对照实验：16 名经验丰富的开源开发者在自己长期维护的仓库使用 AI，实际平均慢了 19%，自己却感觉快了 20%

- ACM CCS 2023 相关研究：使用 AI 助手的参与者写出了更多不安全代码，对结果的信任反而更高

跨端时，不熟悉的语言、系统和发布链路，会让人更难判断自己究竟节省了多少时间。同时，AI 生成代码常常局部看着合理，合到仓库里却出现重复抽象、架构风格漂移、调用链理解不一致。

跑通只是第一步，长期可维护才是更大的挑战。

针对现状，团队已经开始补质量底线：

- **单测与发布卡点**：后端同学 A 建立了 CIS 单测规范，接入发布增量覆盖率卡点，行覆盖率稳定在 **80%** 以上
- **复盘规则前置**：后端同学 B 从 241 份编码类复盘报告中挖出 361 条规则，形成两个稳定性 CR / 编码 Skill，理论故障拦截率达 **70%**

有了结果，效果该怎样验证

智能体应急助手已在多场景上线，但**故障覆盖数、真实诊断时长、建议采纳率**还没有形成稳定口径，MTTR 是否缩短，仍需进一步测量。改 Prompt、换模型、调 Skill 时，也缺少一套能反复比较版本的评测集。

版本能力在不断迭代，诊断是否也更加准确，推理的过程，用户是否更早作出了决定？

谁多做了事，又该怎样被看见

技术问题之外，角色的演变也在追问组织：

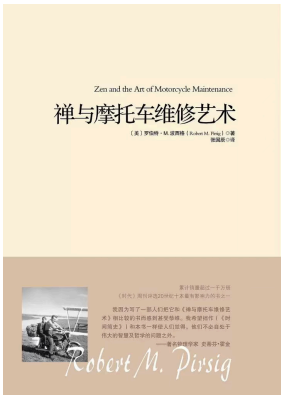
- **PDFE 把产品、设计、前端的事接在一起，业绩怎么算？**
- **跨端是长期安排，还是特殊阶段的补位？**
- **一个人缩短了团队等待，却承担了更多判断，如何衡量他的投入？**

S2，先补未完成的事

已经看到的变化	接下来要验证的事
AI 使用更普遍，产出更多	返工、缺陷与长期维护成本是否改善
原型代码复用率提高，交接减少	非标交互如何交付，移植代码能否持续维护
跨端需求已上线	结对、评审和维护投入是否值得
Skill 被下载，Plugin 被安装	使用者能否完成真实任务，改进能否回流
应急助手能生成建议	诊断是否准确、建议是否被采用、执行是否受控



07
尾声



“约翰的宝马摩托，车把松了，「得垫个垫片——我这儿就有」。我举起手里的啤酒罐，剪一片啤酒罐铝片垫上，就能修好。
约翰拒绝了 —— 他的 BMW 是半个世纪德国机械工艺的骄傲，他无法接受用一片旧啤酒罐来修，宁可车把一直松着。“

—— 我看见的是垫片「意味着什么」；他看见的是垫片「是什么」。

—— 罗伯特·波西格《禅与摩托车维修艺术》(1974)

修好，还是体面？

AI Native 这半年，我们也常站在同一个时刻：AI 写的代码、跨界的角色、拼出来的工具链，看起来都不够“体面”。

我们还是选择了铝片——高效解决当下的问题。Skill、Plugin、PDFE / AI 全栈不是终点，是那片铝片。

参考材料

[1] AI 时代产研组织效能规模化提升实践：
https://mp.weixin.qq.com/s/f5f299W9wxwhKr4PIgFg0Q

[2] 如何把超级个体的产能，转化成组织能力?：
https://mp.weixin.qq.com/s/ywS4Vx2hDdq0BhJbU2CCzw

[3] 从超级个体到超级团队：https://mp.weixin.qq.com/s/GZqTLfeOrfrLgG-R2bOjGw

[4] Agentic coding and persistent returns to expertise：
https://www.anthropic.com/research/claude-code-expertise

[5] Sonar 调查：https://www.sonarsource.com/blog/state-of-code-developer-survey-report-the-current-reality-of-ai-coding/

[6] METR 2025 年对照实验：https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[7] ACM CCS 2023 相关研究：https://dl.acm.org/doi/10.1145/3576915.3623157



欢迎留言一起参与讨论~

AI Native · 目录

上一篇
从榜单走向真实业务负载：Kernel Agent 刷新 Qwen3.8-Max 核心推理算子性能

下一篇
高德本地生活分析 Agent 的 AI Native 实践全景（万字长文回顾）

阅读 2205

留言 1

写留言



Solkatt 浙江 9月29日 4

最近看过最好的AI实践文章。都说前端已经🈚了，看完发现前端居然还能做这么多事情，突然没那么焦虑了

首评